

Pop Quiz #1 문제 풀이

[문제 1]

🔍 Question

하노이 타워 문제를 위한 코드에 대해, 시간복잡도를 나타내는 재귀관계식으로 올바른 것은? (단, n 은 디스크 개수를 나타내는 자연수이다.)

```
def hanoi_tower(n, fr, tmp, to) :  
    if (n == 1) :  
        print("원판 1: %s --> %s" % (fr, to))  
    else :  
        hanoi_tower(n - 1, fr, to, tmp)  
        print("원판 %d: %s --> %s" % (n, fr, to))  
        hanoi_tower(n - 1, tmp, fr, to)
```

📖 풀이 (PDF 원문)

정답: $T(n) = 2T(n-1) + 1$, $T(1) = 1$

강의 자료(p.33) 상세 풀이:

- 순환 관계식: $T(n) = T(n-1) + 1 + T(n-1) = 2T(n-1) + 1$
- 연속 대치법에 의한 풀이:
$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2[2T(n-2) + 1] + 1 = 2^2T(n-2) + 2 + 1 \\ &= 2^3T(n-3) + 2^2 + 2 + 1 \\ &= 2^{n-1}T(1) + 2^{n-2} + \dots + 2^1 + 2^0 \\ &= 2^n - 1 \end{aligned}$$
- 복잡도: $O(2^n)$

☰ 상세 해설

원반 n 개를 옮기는 과정은 (1) 위의 $n-1$ 개를 보조 기둥으로 이동, (2) 바닥 원반을 목적지로 이동, (3) 보조 기둥의 $n-1$ 개를 다시 목적지로 이동하는 세 단계로 나뉩니다. 이것이 $T(n-1) + 1 + T(n-1)$ 이 되어 결과적으로 $2T(n-1) + 1$ 이라는 재귀 식이 만들어집니다.

[문제 2]

🔍 Question

입력의 크기 n 에 대한 시간 복잡도를 나타내는 식 $T(n)$ 에 대한 재귀 관계식이 다음과 같을 때, $T(n)$ 의 점근적 시간 복잡도로 적합한 것은?

$$T(n) = 2T(n/2) + 1$$

📖 풀이 (PDF 원문)

$n = 2^k$ 로 가정 시:

$$\begin{aligned} T(n) &= 2T(n/2) + 1 \\ &= 2(2T(n/2^2) + 1) + 1 = 2^2T(n/2^2) + 2^1 + 2^0 \end{aligned}$$

$$= \dots$$

$$= 2^k T(1) + 2^{k-1} + \dots + 2^1 + 2^0$$

$$= 2^k + 2^k - 1 = 2n - 1$$
 참고: $n < 2^k$ 인 경우에도 점근적 표기에 따르면 $T(n) \in O(n)$ 이라고 쓸 수 있다.

☞ 상세 해설 (이해하기 쉬운 풀이)

1. 재귀 구조의 의미

- $T(n) = 2T(n/2) + 1$ 은 문제를 절반으로 나누어 두 번 처리하고, 추가적으로 상수 시간(+1)의 연산이 필요한 구조입니다. (예: 이진 트리 탐색 등)

2. 재귀 트리(Recursion Tree) 분석

- Level 0:** 1개 노드 (작업량: 1)
- Level 1:** 2개 노드 (작업량: $1 \times 2 = 2$)
- Level 2:** 4개 노드 (작업량: $1 \times 4 = 4$)
- 층이 깊어질수록 각 층의 작업량이 **2배씩** 증가합니다.
- 전체 작업량은 $1 + 2 + 4 + \dots + n$ (등비수열의 합)이며, 이는 약 $2n - 1$ 이 됩니다.

3. 마스터 정리(Master Theorem) 적용

- $T(n) = aT(n/b) + f(n)$ 형태에서 $a = 2, b = 2, f(n) = 1$ 입니다.
- $\log_b a = \log_2 2 = 1$ 이고, $f(n) = n^0$ 이므로 $\log_b a$ 가 더 큽니다.
- 마스터 정리 Case 1에 의해 $T(n) = \Theta(n^{\log_b a}) = \Theta(n^1)$, 즉 $O(n)$ 이 됩니다.

[문제 3]

🔍 Question

0과 1로 구성된 text의 길이가 20이고, 다음과 같이 처음 19개엔 0, 마지막에 1이 있다.

0000...000001

여기서 길이가 5인 패턴 00001을 찾기 위해 수업시간에 배운 억지기법을 사용할 때, 비교연산은 총 몇 번 필요한가?

📖 풀이 (PDF 원문)

처음 5번의 비교 후 실패를 안다.

text: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

pattern: 0 0 0 0 1

이 후 패턴을 한 칸씩 이동하면서 계속 비교하고, 그때마다 5번의 비교 후 실패를 얻는다. 이런 반복을 패턴이 마지막에 발견될 때까지 계속한다.

text: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

pattern: 0 0 0 0 1

즉, 패턴의 길이만큼 비교 후 실패하는 횟수가 15번, 마지막 비교에서 성공하므로 총 16×5 회 비교하게 된다.

☞ 상세 해설

억지 기법(Brute-force)은 패턴을 한 칸씩 밀며 비교합니다. 패턴이 놓일 수 있는 총 위치는 $20 - 5 + 1 = 16$ 곳입니다. 텍스트가 패턴의 마지막 글자만 다른 형태(00001)이므로, 매 위치마다 패턴 길이인 5번의 비교를 모두 수행해야 불일치를 알 수 있습니다. 따라서 $16 \times 5 = 80$ 번의 비교가 발생합니다.

[문제 4]

🔍 Question

어떤 함수 $f(n) = O(n^2)$ 일 때, 다음 중 $f(n)$ 이 될 수 없는 것은?

- ① $n^2 + 2^n$
- ② $n \lg n + \lg n^3$
- ③ $3n$
- ④ $4n + (\lg n)^3$
- ⑤ $100n^2 + 5n$

📖 풀이 (PDF 원문)

정답: ①

최고차항이 2^n 이므로 $2^n \notin O(n^2)$ 이다. 나머지는 모두 성립한다.

[문제 5]

🔍 Question

어떤 함수 $f(n) = \Omega(n \lg n)$ 일 때, 다음 중 $f(n)$ 이 될 수 없는 것은?

- ① $\lg^2 n$
- ② $\lg(n!)$
- ③ $\lg n^3 + n^2$
- ④ n^2
- ⑤ $n\sqrt{n}$

📖 풀이 (PDF 원문)

정답: ①

$f(n)$ 의 최고차항이 $n \lg n$ 이상이어야 한다. 이 조건을 만족하지 않는 식은 ①이다.

🔄 복잡도의 위계 (Complexity Hierarchy)

$1 \ll \log \log n \ll \log^k n \ll \sqrt[k]{n} \ll n \ll n \log^k n \ll n\sqrt{n} \ll n^2 \ll n^i \ll n^j \ll a^n \ll b^n \ll n! \ll n^n$
(단, $i < j, 1 < a < b$)

- $\log(n!) \in \Theta(n \log n)$

☰ 상세 해설

- $O(n^2)$ (상한): 함수의 증가율이 n^2 을 넘지 않아야 합니다. 2^n 은 지수적으로 증가하므로 $O(n^2)$ 에 포함될 수 없습니다.
- $\Omega(n \lg n)$ (하한): 함수의 증가율이 적어도 $n \lg n$ 이상이어야 합니다. $\lg^2 n$ 은 $n \lg n$ 보다 차수가 낮으므로 하한 조건을 만족하지 못합니다.

🔄 점근적 표기법 핵심 요약

- $O(g(n))$ (Big-O): 상한(Upper Bound). 함수의 증가율이 $g(n)$ 보다 작거나 같음 ($f(n) \leq c \cdot g(n)$). "최악의 경우에도 이 보다는 빠르다."
- $\Omega(g(n))$ (Big-Omega): 하한(Lower Bound). 함수의 증가율이 $g(n)$ 보다 크거나 같음 ($f(n) \geq c \cdot g(n)$). "최소한 이 정도의 성능은 보장한다."

- $\Theta(g(n))$ (**Big-Theta**): 딱 맞는 한계(**Tight Bound**). 상한과 하한이 같음 (O 이면서 동시에 Ω). "정확히 이 정도의 차수로 증가한다."

True/False 문항별 상세 풀이

② $\lg(n!) \in \Theta(n \ln n) \rightarrow \text{True}$

스털링 근사($n! \approx \sqrt{2\pi n}(n/e)^n$)에 의해 $\lg(n!) \approx n \lg n$ 이 성립합니다. $\ln n$ 과 $\lg n$ 은 상수배 차이($\log_a x = \frac{\log_b x}{\log_b a}$)일 뿐이므로 차수가 동일한 Θ 관계가 맞습니다.

② $n + 10 \in O(n^2) \rightarrow \text{True}$

상한(O) 비교입니다. 좌변은 1차식(n)이고 우변은 2차식(n^2)입니다. 낮은 차수의 함수는 높은 차수의 함수를 상한으로 가질 수 있으므로 참입니다.

② $(\lg n)^2 \in \Theta(\sqrt{n}) \rightarrow \text{False}$

로그 함수의 거듭제곱($(\lg n)^2$)은 아무리 지수가 커도 다항 함수($\sqrt{n} = n^{0.5}$)보다 성장이 느립니다. 따라서 두 함수의 증가율이 같은 Θ 관계는 성립할 수 없습니다.

② $n^n + \ln n \in O(n!) \rightarrow \text{False}$

복잡도의 위계에서 n^n 은 $n!$ 보다 훨씬 빠르게 증가하는 함수입니다. 더 큰 함수를 더 작은 함수($n!$)로 상한(O) 지을 수 없으므로 거짓입니다.

② $2^n \in \Omega(3^n) \rightarrow \text{False}$

하한(Ω) 비교입니다. 2^n 은 3^n 보다 성장이 느린 함수입니다. 2^n 이 최소한 3^n 만큼의 속도로 증가한다는 의미의 $\Omega(3^n)$ 은 성립하지 않습니다.

② 모든 tree는 그래프이다. $\rightarrow \text{True}$

트리의 정의는 '사이클이 없고 연결된 무방향 그래프'입니다. 따라서 모든 트리는 그래프의 부분 집합에 해당합니다.

② 정점의 개수가 n 인 tree에 있는 에지의 개수는 $(n - 1)$ 개이다. $\rightarrow \text{True}$

임의의 트리가 n 개의 정점을 가질 때, 모든 정점을 사이클 없이 최소한으로 연결하기 위한 에지의 개수는 항상 $n - 1$ 개로 고정됩니다.

② 정점을 잇는 에지들로부터 사이클을 찾을 수 없는 graph는 tree이다. $\rightarrow \text{False}$

사이클이 없다는 조건만으로는 부족합니다. 그래프가 연결(**Connected**)되어 있어야 트리입니다. 사이클은 없지만 연결되지 않은 그래프는 여러 개의 트리로 구성된 포레스트(**Forest**)입니다.